PROJECT REPORT

SMART STUDENT RECORD FINDER USING BINARY SEARCH

Submitted by

Name: Johan R Mathew
Roll Number: 17

Course Code: 24CCT301

Department: Computer Science and Engineering (Cyber Security)

SMART STUDENT RECORD FINDER USING BINARY SEARCH

1. Introduction

Searching is an important operation in computer applications, especially when retrieving a particular student's details from a collection of records. When the number of records increases, an efficient searching algorithm can reduce the time required to find information.

The Smart Student Record Finder is a GUI-based application developed using Python and Tkinter. It uses Binary Search to locate student records based on their roll numbers. The application maintains records in sorted order and displays the searching process through an animated visualization.

The system also provides a comparison between Linear Search and Binary Search, helping users understand the difference between the two algorithms in terms of the number of comparisons.

2. Problem Statement

Searching for a student manually in a large collection of records can be time-consuming. It may also be difficult for students to understand how searching algorithms work when they are explained only through theory or pseudocode.

Therefore, a graphical application is developed to search student records efficiently and demonstrate the working of Binary Search through animation. The application allows users to add student records, search by roll number, observe the algorithm's progress, and compare the performance of Linear Search and Binary Search.

3. Objectives

- To implement Binary Search using Python.
- To develop a user-friendly graphical interface using Tkinter.
- To maintain student records in ascending order of roll number.
- To demonstrate Binary Search through step-by-step animation.
- To provide Start Animation and Pause/Resume controls.
- To allow users to adjust the animation speed.

- To compare Linear Search and Binary Search based on the number of comparisons.
- To study the time and space complexity of both algorithms.

4. Proposed System

The proposed system is a desktop application that manages student records and demonstrates searching algorithms visually.

The user can enter a student's roll number, name, and marks. The application validates the input and adds the record to the student list. Records are arranged in ascending order of roll number so that Binary Search can be performed correctly.

When the user searches for a roll number, the application examines the middle element of the current search range. Depending on the comparison, it eliminates the unnecessary half and continues searching.

The animated visualization uses different colours to represent the algorithm's progress:

- **Blue:** The middle element currently being examined.
- **Green:** The student record successfully found.
- **Grey:** Records outside the active search range.

The user can start the animation, pause or resume it, and change the delay between steps. The application also compares Linear Search and Binary Search by displaying their search results and comparison counts.

5. Implementation Work

5.1 Technologies Used

- Programming Language: Python
- GUI Library: Tkinter
- GUI Styling: ttk
- Development Environment: Visual Studio Code
- Data Structure: Python list of student records

5.2 Student Record Management

Each student record contains three fields: roll number, student name, and marks.

The application checks that the roll number is valid and unique, the name is not empty, and marks are between 0 and 100. Student records are sorted by roll number after insertion.

The records are stored in memory during execution. Permanent database storage is not included in the current version.

5.3 Binary Search Algorithm

Binary Search is an efficient searching algorithm that works on sorted data. It repeatedly compares the target value with the middle element of the current search range.

Algorithm: Binary Search

1. Start with the first and last positions of the sorted list.
2. Calculate the middle position.
3. Compare the middle element's roll number with the target roll number.
4. If both values are equal, report that the student is found.
5. If the target is smaller, continue searching the left half.
6. If the target is larger, continue searching the right half.
7. Repeat until the student is found or the search range becomes empty.
8. If the range becomes empty, report that the student is not found.

5.4 Pseudocode

```
BINARY_SEARCH (records, target)

  low = 0
  high = length(records) - 1

  while low <= high

    mid = (low + high) // 2

    if records[mid].roll == target
      return mid

    else if target < records[mid].roll
      high = mid - 1

    else
      low = mid + 1

  return NOT_FOUND
```

5.5 Animated Visualization

The animated visualization demonstrates the search process one comparison at a time.

1. The user enters a target roll number.
2. The user clicks **Start Animation**.
3. The application highlights the middle element in blue.
4. The comparison result determines which half of the search range is eliminated.
5. The visualization updates to show the reduced search range.
6. The process repeats until the target is found or no elements remain.
7. The final result and comparison count are displayed.

The user can pause the animation and resume it later. The speed control allows the user to select a delay of 1500 milliseconds, 900 milliseconds, or 400 milliseconds between steps.

A separate **Next Step** control is also available for manually advancing the search process.

5.6 Linear Search and Binary Search Comparison

The application includes a comparison feature that executes both algorithms for the same target roll number.

Linear Search examines records one by one until the target is found or the list ends. Binary Search repeatedly divides the sorted search range into two halves.

The comparison window displays whether the target was found and the number of comparisons performed by each algorithm.

5.7 GUI Components

- **Header:** Displays the project title and subject.
- **Summary Cards:** Display the total number of students, comparison count, and search status.
- **Add Student Record:** Accepts roll number, name, and marks.
- **Find Student:** Provides the search field and search controls.
- **Animation Controls:** Provides Start Animation, Pause/Resume, and speed selection.
- **Binary Search Visualization:** Displays the current middle element and search range.
- **Student Records Table:** Displays all stored student records.
- **Status Bar:** Displays progress messages, search results, and comparison information.

6. Algorithm Analysis

6.1 Binary Search Complexity

Let (n) represent the number of student records.

Case	Time Complexity	Explanation
Best case	(O(1))	The target is at the middle position during the first comparison.

Average case	(O(\log n))	The search range is repeatedly reduced by half.
Worst case	(O(\log n))	The range is repeatedly divided until the target is found or the range becomes empty.

Auxiliary Space Complexity: (O(1)), because the iterative algorithm uses a constant number of additional variables.

6.2 Linear Search Complexity

Case	Time Complexity	Explanation
Best case	(O(1))	The target is the first record examined.
Average case	(O(n))	Several records may need to be examined.
Worst case	(O(n))	Every record may need to be examined.

Auxiliary Space Complexity: (O(1)), because the iterative search uses a constant amount of additional space.

6.3 Additional Processing

The complexity values describe the searching algorithms themselves. The application also performs input validation, maintains sorted records, and redraws the GUI during animation. These operations introduce additional processing beyond the core search algorithm.

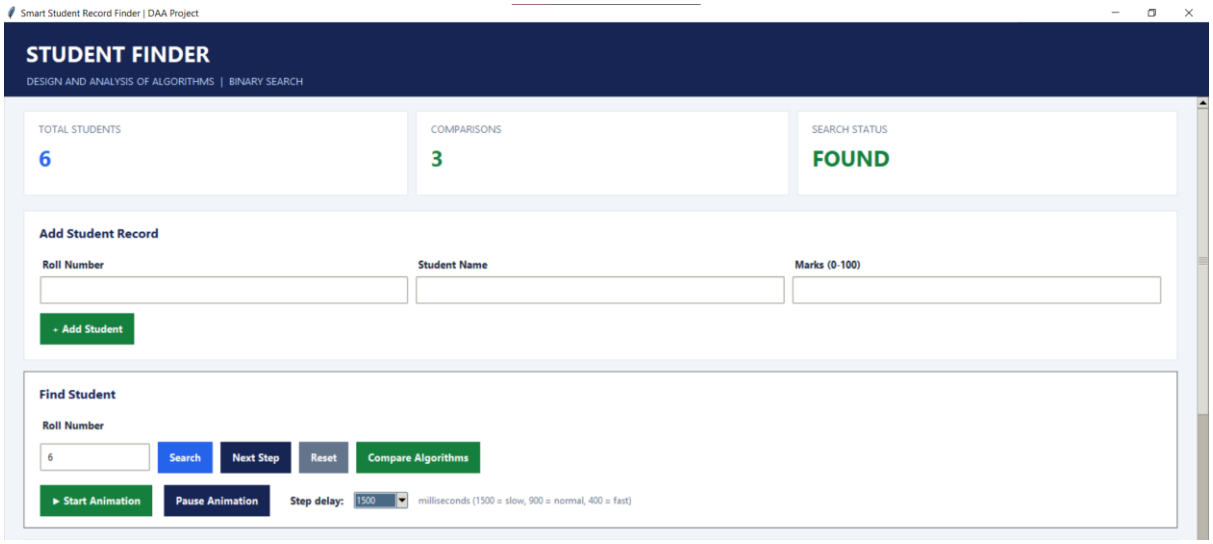
7. Testing and Results

Execute the following test cases and record the actual results after running the application.

Test Case	Input or Action	Expected Result
TC01	Add a valid student record	Record appears in the table.
TC02	Add a duplicate roll number	An error message is displayed.
TC03	Enter marks below 0 or above 100	Invalid marks are rejected.
TC04	Search for an existing roll number	The correct student is found.
TC05	Search for a missing roll number	A not-found message is displayed.
TC06	Start the animation	Search steps are shown automatically.
TC07	Pause and resume the animation	Animation pauses and continues when requested.
TC08	Change the animation delay	The interval between steps changes.
TC09	Compare both algorithms	Search results and comparison counts are displayed.
TC10	Search when there are no records	A warning is displayed.

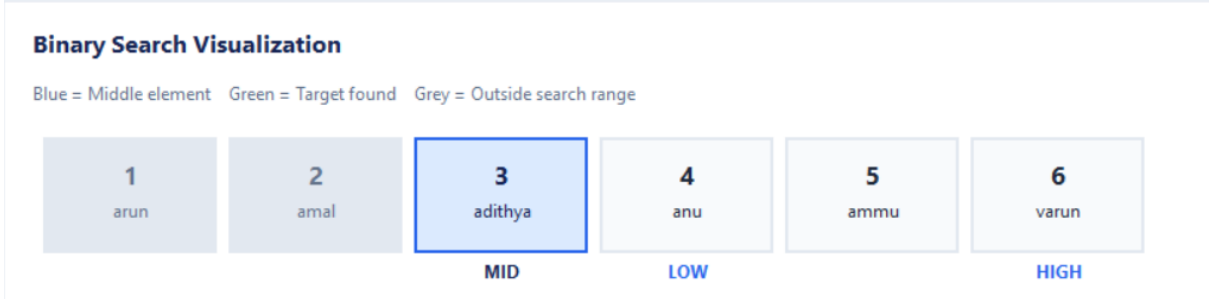
Results: Fill in the actual outcome of each test after verifying it.

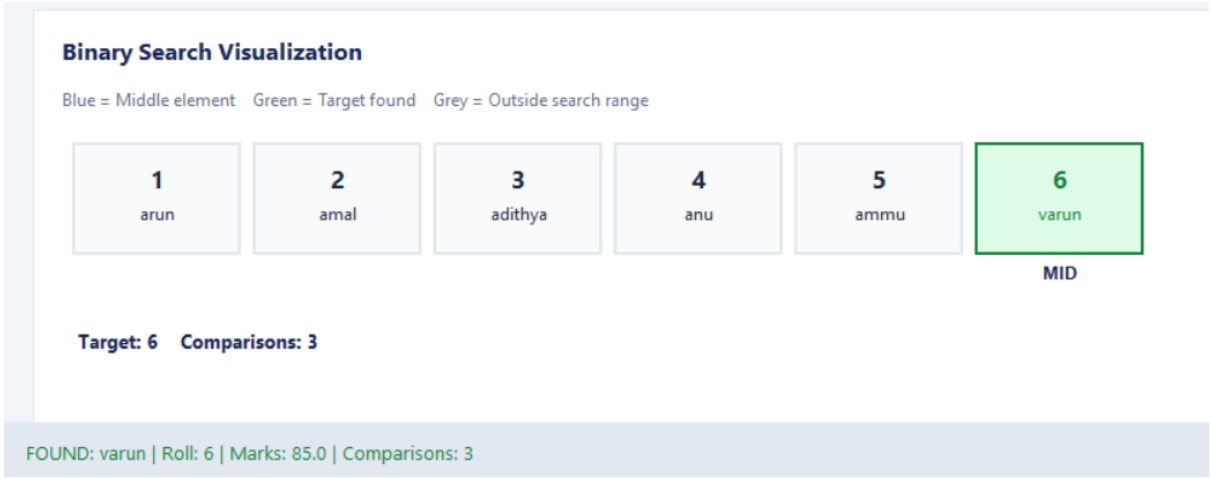
8. GUI Screenshots

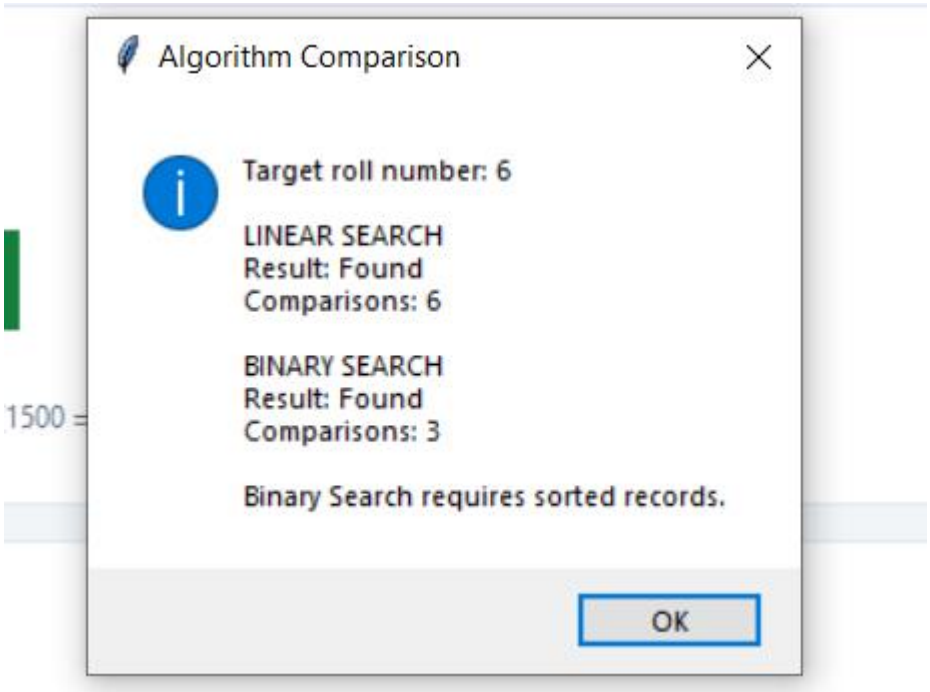












9. Advantages

- Provides a user-friendly graphical interface.
- Demonstrates Binary Search visually.
- Allows users to control animation speed and pause or resume execution.
- Maintains records in sorted order for Binary Search.
- Displays the number of comparisons performed.
- Helps users understand the difference between Linear Search and Binary Search.
- Validates student information before adding records.

10. Limitations

- Student records are stored in memory and are lost when the application closes.

- Permanent database storage is not included.
- Dedicated update and delete operations are not included in the current GUI.
- Binary Search requires records to be sorted by roll number.
- Linear Search does not have a separate animated visualization.

11. Future Enhancements

- Integrate MySQL or SQLite for permanent storage.
- Add update and delete operations for student records.
- Provide a separate animated visualization for Linear Search.
- Display a graphical comparison of the number of comparisons made by both algorithms.
- Support exporting student records to CSV or Excel.
- Add sorting options and search by additional student attributes.

12. Conclusion

The Smart Student Record Finder using Binary Search demonstrates how an efficient searching algorithm can be implemented in a practical GUI-based application. Developed using Python and Tkinter, the system allows users to add student records, search by roll number, and observe the search process through animation.

The Start Animation, Pause/Resume, and speed-control features make the algorithm easier to understand. The comparison feature also demonstrates the difference between Linear Search and Binary Search in terms of the number of comparisons performed.

This project provides practical experience in algorithm implementation, complexity analysis, GUI development, and visualization of algorithm execution. Future improvements can include database integration, complete record-management operations, and animation for additional searching algorithms.